
SSL, TLS and DTLS Protocols

5.1. Introduction

Secure Sockets Layer (SSL) and Transport Layer Security (TLS) protocols are used to create a secure session between a host, which initializes the session, and a security gateway acting as a server, localized in the protected Local Area Network (LAN).

SSL/TLS protocols provide an alternative solution to the Internet Protocol Security (IPSec) mechanism described in Chapter 4.

Secure data transport is implemented between a client (the host) initializing the session and a server (the security gateway), localized in the LAN network protected by SSL/TLS protocols.

The TLS protocol is standardized by the Internet Engineering Task Force (IETF). It succeeds the SSL protocol developed by the Netscape, the original purpose of which was to secure exchanges between a navigator and a web server.

SSL/TLS protocols correspond to a *Record* header inserted between the transport layer and the messages of the

application layer, or the following SSL/TLS messages (Figure 5.1):

- the *change_cipher_spec* message indicates a modification of security parameters;
- the *alert* message indicates an error in communication between the host and the security gateway;
- the *handshake* messages negotiate the security parameters between the host and the security gateway.

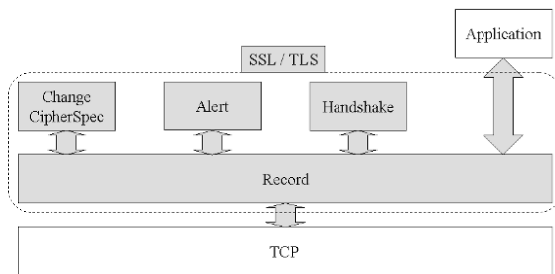


Figure 5.1. *SSL / TLS protocol architecture*

SSL/TLS protocols impose a reliable transport protocol (for example, Transmission Control Protocol (TCP)), which enables the exchange of messages without errors or risks of desequencing.

SSL/TLS protocols delimit messages determining the start and end of each message. Message delimitation is obtained via an indication of length contained in the message. This also enables the transport of multiple SSL/TLS messages in a TCP segment.

The Datagram TLS (DTLS) protocol reuses the main functionalities of SSL/TLS protocols. The modifications it contributes arise from the use of User Datagram Protocol (UDP), Datagram Congestion Control Protocol (DCCP), Stream Control Transmission Protocol (SCTP) and Secure Real-time Transport Protocol (SRTP).

5.2. SSL/TLS protocols

5.2.1. *Record header*

The *Record* header encapsulates SSL/TLS messages or messages originating from the application layer. These messages can be completely encrypted, checked and authenticated due to a seal contained in the Message Authentication Code (MAC) field.

The structure of the *Record* header is shown in Figure 5.2.

The *Protocol* field, coded on 1 byte, indicates the type of message encapsulated:

- value = 20 for the *change_cipher_spec* message;
- value = 21 for the *alert* message;
- value = 22 for the *handshake* messages;
- value = 23 for the message generated by the application.

The *Version* field, coded on 2 bytes, indicates the two values of the protocol version. The latest version of the SSL protocol has a value of {3,0}. The first version of the TLS 1.0 protocol has a value of {3,1}. The current version of the TLS 1.2 protocol has a value of {3,3}.

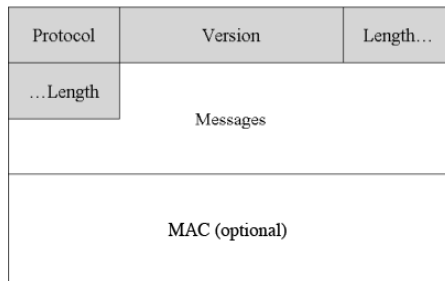


Figure 5.2. *Record header of SSL/TLS protocol*

The *Length* field, coded on 2 bytes, indicates the size of the data encapsulated. This size cannot exceed 2^{14} bytes.

5.2.2. *Change_cipher_spec* message

The *change_cipher_spec* message is simple and defines a single type of message indicating that a new key and a new set of algorithms are being used for message protection. The *change_cipher_spec* message includes a field coded on 1 byte with a value of 1 (Figure 5.3).

Protocol = 20	Version		Length = 0
Length = 1	change cipher_spec = 1		

Figure 5.3. *Change_cipher_spec* message

5.2.3. *Alert* message

The *alert* message is coded on 2 bytes and contains the *Level* and *Description* fields, with each field coded on 1 byte (Figure 5.4).

Protocol = 21	Version		Length = 0
Length = 2	Level	Description	

Figure 5.4. *Alert* message

The *Level* field indicates the level of severity of the alarm. Two values are defined:

- a value of 1 indicates a minor alarm. The two parties can continue the current session, but they are not required to establish a new transaction;

– a value of 2 indicates a major alarm. The two parties must end the current session.

The *Description* field indicates the type of alarm encountered. Table 5.1 gives a list of the alarms defined for the SSL protocol.

Name	Value	Description
<i>close_notify</i>	0	This minor alarm tells the receiver not to send any more messages for the current transaction.
<i>unexpected_message</i>	10	This major alarm indicates that an inappropriate message has been received.
<i>bad_record_mac</i>	20	This minor alarm indicates the reception of a <i>Record</i> message with an incorrect MAC seal value.
<i>decompression_failure</i>	30	This major alarm indicates that data decompression has failed.
<i>handshake_failure</i>	40	This major alarm indicates that the <i>handshake</i> procedure has failed.
<i>no_certificate</i>	41	This minor alarm is transmitted by the client to indicate that it does not possess the certificate required by the server.
<i>bad_certificate</i>	42	This minor alarm indicates that the certificate received is corrupted.
<i>unsupported_certificate</i>	43	This minor alarm indicates that the certificate received is not supported.
<i>certificate_revoked</i>	44	This minor alarm indicates that the certificate received has been revoked.
<i>certificate_expired</i>	45	This minor alarm indicates that the certificate is no longer valid.
<i>certificate_unknown</i>	46	This minor alarm indicates that a problem has occurred with the certificate.
<i>illegal_parameter</i>	47	This major alarm indicates that a field has a prohibited value during the <i>handshake</i> procedure.

Table 5.1. Alarms defined by the SSL protocol

The TLS protocol has removed the *no_certificate* alarm and added the list of alarms shown in Table 5.2.

Name	Value	Description
<i>record_overflow</i>	22	This major alarm indicates that the size of the <i>Record</i> message is too large.
<i>unknown_ca</i>	48	This major alarm indicates that the certification authority is unknown.
<i>access_denied</i>	49	This major alarm indicates that access is refused even though a valid certificate has been received.
<i>decode_error</i>	50	This major alarm indicates that the message cannot be interpreted.
<i>decrypt_error</i>	51	This major alarm indicates that the authentication operation has failed.
<i>protocol_version</i>	70	This major alarm indicates that the protocol version is not supported.
<i>insufficient_security</i>	71	This major alarm indicates that the <i>handshake</i> procedure has failed because the client algorithms do not provide adequate security.
<i>internal_error</i>	80	This major alarm indicates that an internal error has interrupted the <i>handshake</i> procedure.
<i>user_canceled</i>	90	This minor alarm indicates that the transaction has been terminated.
<i>no_renegotiation</i>	100	This minor alarm is transmitted to indicate rejection of a new negotiation.
<i>unsupported_extension</i>	110	This major alarm is transmitted by the client if the server proposal in the <i>server_hello</i> message is different than the proposals initiated by the client.

Table 5.2. Alarms added by the TLS protocol

5.2.4. Handshake messages

The *handshake* message fulfills the following functions:

- the parties (client and server) exchange *hello* messages to define algorithms and transmit random numbers;

- the parties exchange cryptographic parameters enabling them to build the PreMaster secret;
- the parties exchange certificates and cryptographic parameters enabling their mutual authentication;
- the parties generate the Master key from the PreMaster secret;
- the parties confirm that they have calculated the same security parameters.

The *handshake* messages start with the *Type* field, coded on 1 byte and indicating the type of message, and the *Message Length* field, coded on 3 bytes and indicating the message size excluding the *Type* and *Message Length* fields.

Table 5.3 shows the value of the *Type* field for the different *handshake* messages.

<i>Type</i> field	Message
1	<i>client_hello</i>
2	<i>server_hello</i>
11	<i>server_certificate</i>
12	<i>server_key_exchange</i>
13	<i>certificate_request</i>
14	<i>server_hello_done</i>
15	<i>certificate_verify</i>
16	<i>client_key_exchange</i>
20	<i>finished</i>

Table 5.3. *Different handshake messages*

The *client_hello* and *server_hello* messages insert the protocol version number in the *Protocol Version* field. Though this field appears in the header of the *Record* protocol, it enables the *Record* protocol and the *handshake* messages to evolve independent of one another.

The *handshake* procedure is described in section 2.5.2 concerning the EAP-TLS procedure.

5.2.4.1. *Hello_request message*

The *hello_request* message is transmitted by the server to initialize the *handshake* procedure. The *hello_request* message is a special message containing no fields.

5.2.4.2. *Client_hello message*

The *client_hello* message is used by the client to start the transaction. This message is transmitted upon reception of the *hello_request* message or at the client's initiative in order to renegotiate security parameters.

The structure of the *client_hello* message is shown in Figure 5.5.

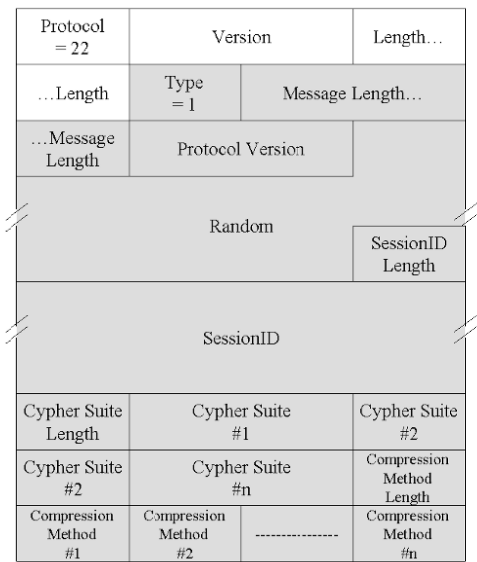


Figure 5.5. *Client_hello message*

The *Random* field, coded on 32 bytes, contains a random number. The first four bytes contain the date and time down to the second, which makes it possible to avoid duplication of this field's value.

The *SessionID* Length field, coded on 1 byte, indicates the size of the SessionID field.

The *SessionID* field contains the value of the identifier of the session the client wishes to establish. This field is empty if the client wishes to negotiate new security parameters. The client fills this field with the value communicated by the server if it wishes to recover a previous session.

The *Cipher Suite Length* field, coded on 1 byte, indicates the total size of the *Cipher Suite* fields containing proposed algorithmic series. Each *Cipher Suite* field is coded on 2 bytes.

Table 5.4 shows the algorithms proposed by SSL and TLS protocols.

Security functions	SSL	TLS
Key establishment	RSA DH-RSA DH-DSS DHE-RSA DHE-DSS DH-Anon Fortezza-KEA	RSA DH-RSA DH-DSS DHE-RSA DHE-DSS DH-Anon
Encryption	IDEA-CBC RC4-128 3DES-EDE-CBC Fortezza-CBC	IDEA-CBC RC4-128 3DES-EDE-CBC Kerberos AES
Signature	RSA DSA	RSA DSA EC
Hashing	MD5 SHA-1	MD5 SHA-1

Table 5.4. *Algorithmic suites*

The *Compression Method Length* field, coded on 1 byte, indicates the total size of the *Compression Method* fields containing proposed compression methods. Each

Compression Method field is coded on 1 byte. No compression method is currently defined if:

- the value of the *Compression Method Length* field is 1;
- the value of the *Compression Method* field is 0.

The *client_hello* message can request extended functionalities following the list of compression methods from the server. If these functionalities are not available from the server, the *handshake* procedure must be abandoned.

5.2.4.3. *Hello_server message*

The *hello_server* message is the server's response to the *hello_client* message received. The *hello_server* message has a structure similar to the *hello_client* message. The server chooses an algorithm suite and a compression method from the lists proposed by the client.

The server can indicate a value in the *SessionID* field. In this case, the client is authorized to recover this session subsequently. In the opposite case, this field has a value of zero.

5.2.4.4. *Certificate message*

The *certificate* message contains a hierarchical list of certificates:

- the first certificate is the transmitter's;
- the next certificate is the certificate of the authority certifying the previous certificate;
- the third certificate, if there is one, is the authority's certification.

The structure of the *certificate* message is shown in Figure 5.6.

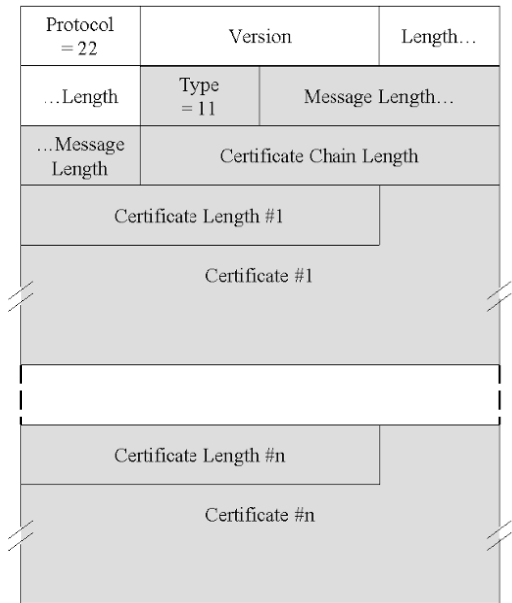


Figure 5.6. *Certificate message*

The value of the *Certificate Chain Length* field, coded on 3 bytes, is equal to the value of the *Message Length* field reduced by 3 bytes.

The *Certificate Length* field, coded on 3 bytes, indicates the size of the certificate.

If the client has produced a list of message signature algorithms in the *client_hello* message extension, the server must sign the list of certificates provided.

5.2.4.5. *Server_key_exchange message*

The *server_key_exchange* message transports cryptographic information from the server to the client, which will enable the client to generate and transmit the PreMaster secret.

Cryptographic information depends on the way in which the key is generated. The structure of the *server_key_exchange* message, shown in Figure 5.7, corresponds to a Diffie–Hellman exchange.

Protocol = 22	Version		Length...
...Length	Type = 12	Message Length...	
...Message Length	dh_p Length		dh_p...
....dh_p		dh_g Length	
....dh_g		dh_Ys Length	
dh_Ys			

Figure 5.7. *Server_key_exchange* message

The *dh_p* field contains the value of the prime number p . The length of this field is indicated by the *dh_p Length* field.

The *dh_g* field contains the value of the generator g . The length of this field is indicated by the *dh_g Length* field.

The *dh_Ys* field contains the public Diffie–Hellman value of the server. The length of this field is indicated by the *dh_Ys Length* field.

The *server_key_exchange* message can also include a signature after the *dh_Ys* field. The signature’s parameters depend on the algorithm indicated in the server’s certificate.

5.2.4.6. *Certificate_request* message

The *certificate_request* message is transmitted by the server to request the client’s certificate. The

certificate_request message also indicates the appropriate certification authorities.

The structure of the *certificate_request* message is shown in Figure 5.8.

The *Certificate Type Length* field, coded on 1 byte, indicates the total size of the group of *Certificate Type* fields.

The *Certificate Type* field, coded on 1 byte, provides the type of certificate:

- value = 1: RSA, for key exchange signature;
- value = 2: DSA, for signature only;
- value = 3: RSA, for signature with Diffie–Hellman fixed-key exchange;
- value = 4: DSA, for signature with Diffie–Hellman fixed-key exchange;
- value = 5: RSA, for signature with Diffie–Hellman temporary-key exchange;
- value = 6: DSA, for signature with Diffie–Hellman temporary-key exchange;
- value = 20: Fortezza/DMS, for signature and key exchange.

The *Certificate Authorities Length* field, coded on 2 bytes, contains the total size of the group of *Certificate Authorities* fields. Each *Certificate Authorities* field is also preceded by the *Certificate Authorities Length* field indicating its size.

The *Certificate Authorities* field contains *Distinguished Name* information concerning the certification authority contained in the certificate.

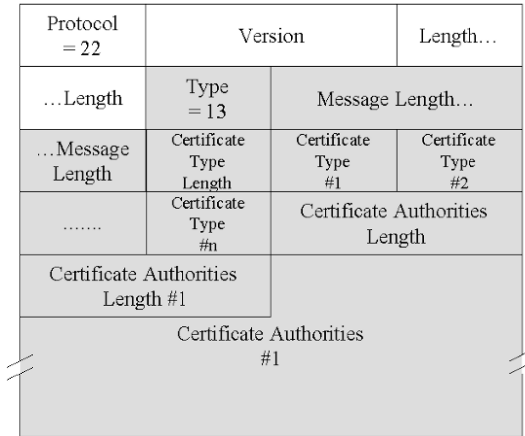


Figure 5.8. *Certificate_request* message

5.2.4.7. *Server_hello_done* message

The *server_hello_done* message is transmitted by the server to indicate the end of the *server_hello* message and the associated *certificate*, *server_key_exchange* and *certificate_request* messages.

5.2.4.8. *Client_key_exchange* message

The *client_key_exchange* message is transmitted by the client just after the *certificate* message containing the client's certificate. The *client_key_exchange* message is used to establish the Premaster secret.

The structure of the *client_key_exchange* message is shown in Figure 5.9 if this secret is established using Diffie–Hellman exchanges.

The *dh_Yc* field contains the client's public Diffie–Hellman value. The length of this field is indicated by the *dh_Yc Length* field. This field appears when the value is temporary. If the value is fixed, it appears in the client's certificate, and in this case the field is empty.

Protocol = 22	Version		Length...
...Length	Type = 16	Message Length...	
Message Length	dh_Yc Length		
dh_Yc			

Figure 5.9. *Client_key_exchange message*

5.2.4.9. *Certificate_verify message*

The *certificate_verify* message is transmitted by the client to prove that it holds the private key corresponding to the public key of its certificate. The *certificate_verify* message contains the signature of a digest calculated from all of the *handshake* messages exchanged since the *client_hello* message and from the Master secret derived from the PreMaster secret.

Calculation of the digest depends on the type of protocol: SSL or TLS.

5.2.4.10. *Finished message*

After the *certificate_verify* message, the client transmits the *change_cipher_spec* message to indicate that the next message, the *finished* message, uses the algorithm suite and key negotiated for the encryption and integrity control functions.

The *finished* message enables the client to terminate the negotiation. It contains the following encrypted information:

- the digest calculated from all of the *handshake* messages exchanged since the *client_hello* message, a

sequence of characters depending on the role of the party (client or server), and the Master secret;

- the MAC seal enabling integrity control and authentication of the *finished* message.

Calculation of the digest depends on the type of protocol: SSL or TLS.

5.2.5. Cryptographic information

5.2.5.1. Key generation

The Master key is derived from the PreMaster key, the implementation of which uses one of the following operations:

- the client and server exchange public Diffie–Hellman values. This mechanism is described in section 5.2.4 concerning *handshake* messages.

- the client generates the PreMaster secret and transmits it to the server, encrypting it using the server’s public key (RSA or Fortezza/DMS mechanisms).

For the SSL protocol, Master key is calculated using message digest 5 (MD5) and Secure Hash Algorithm (SHA) for the calculation of digests.

```
master secret =  
MD5(premaster secret || SHA(ASCII character ‘A’ ||  
premaster secret || client_hello.random ||  
server_hello.random)) ||  
MD5(premaster secret || SHA(ASCII characters ‘BB’ ||  
premaster secret || client_hello.random ||  
server_hello.random)) ||  
MD5(premaster secret || SHA(ASCII characters ‘CCC’ ||  
premaster secret || client_hello.random ||  
server_hello.random))
```


For the TLS protocol, the Master key is calculated using the Hashed message authentication code (HMAC) operation for the calculation of digests.

The next formula is the one defined for the TLS 1.2 protocol.

It differs from the formula used for previous versions, which used a combination of two SHA and MD5 algorithms.

The number of iterations depends on the required size of cryptographic information to be produced.

```

master secret =
HMAC-SHA-256(premaster secret, A(1) || label || seed) ||
HMAC-SHA-256 (secret, A(2) || label || seed) ||
HMAC-SHA-256 (secret, A(3) + label || seed) + ...
with
A(0) = label || seed
A(i) = HMAC(secret, A(i-1))

```

The client and the server then derive the Master secret to generate the following keys (key materials):

- *client write MAC secret*: this secret is used by the client to calculate the MAC seal associated with a message;
- *server write MAC secret*: this secret is used by the server to calculate the MAC seal associated with a message;
- *client write encryption key*: this key is used by the client to encrypt the message and associated MAC seal;
- *server write encryption key*: this key is used by the server to encrypt the message and associated MAC seal;
- *client write IV*: this structure is used by the client as an initialization vector for the encryption algorithm;
- *server write IV*: this structure is used by the server as an initialization vector for the encryption algorithm.

The calculation mechanism is identical to the mechanism used to derive the PreMaster key.

5.2.5.2. *Integrity checking*

For the SSL protocol, the MAC seal is calculated by applying the MD5 algorithm to the following concatenated data:

- the *write MAC* secret, derived from the Master secret;
- byte 36 in hexadecimal notation, repeated 48 times;
- the number of messages exchanged (*Sequence Number*). The counter value is set at zero for each *change_cipher_spec* message, and then increased by 1 unit for each message;
- the *Protocol* and *Length* fields of the *Record* header;
- the message being subjected to integrity checking.

For the TLS protocol, the MAC seal calculation is shown in Figure 5.10.

5.3. DTLS protocol

5.3.1. *Adaptation to UDP transport*

5.3.1.1. *Record header*

The TLS protocol implicitly maintains a message sequence number that is never transmitted. However, this number is used for calculating the MAC seal. The TCP protocol ensures the delivery of segments in sequence. Since this function does not exist for the UDP protocol, message numbering has been introduced by the DTLS protocol.

The structure of the *Record* header is shown in Figure 5.11.

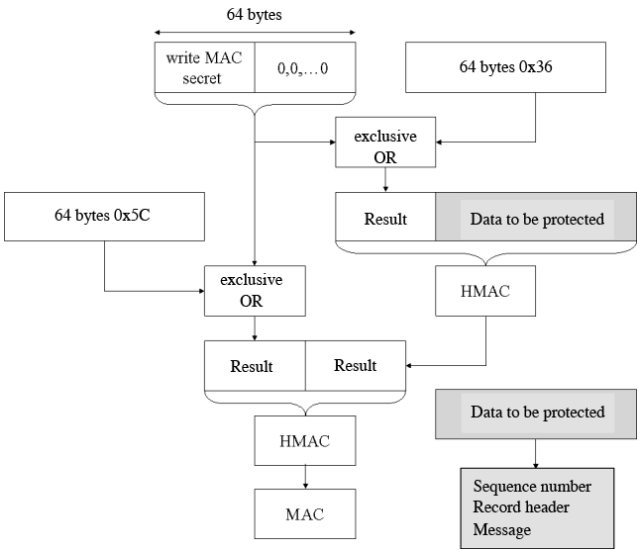


Figure 5.10. Seal calculation for TLS protocol

Protocol	Version	Epoch...
...Epoch	Sequence Number....	
Sequence Number....		Length...
...Length	Messages	
MAC (optional)		

Figure 5.11. Record header of DTLS protocol

The *Epoch* field, coded on 2 bytes, is increased by 1 unit for each successful *handshake* procedure.

The *Sequence Number* field, coded on 6 bytes, is increased by 1 unit for each *Record* message.

5.3.1.2. *Handshake messages*

As for the TLS protocol, the *handshake* procedure starts when the *client_hello* message is sent. Since there is no connection establishment procedure by the transport protocol, there is a risk of denial of service type attack. An attacker can send *client_hello* messages and thus open multiple sessions with the server.

To avoid this problem, the DTLS protocol introduces a new *hello_verify_request* message containing a reference (*cookie*), transmitted by the server in response to the *client_hello* message.

The client must repeat the *client_hello* message, integrating the reference (*cookie*) received after the *SessionID* field. The server can then verify that the client has not usurped its address.

The TCP protocol segments messages that are too long in order to adapt itself to the authorized size. Since this function does not exist for the UDP protocol, the DTLS protocol fragments *handshake* messages so that this operation will not take place in the Internet Protocol (IP) layer.

The DTLS protocol must determine the Maximum Transmission Unit (MTU) size of the IP packet on the route between the client and the server in order to deduce the maximum size of the fragment from it.

The TCP protocol ensures retransmission in case of segment loss. Since this function does not exist for the UDP protocol, the DTLS protocol must arm a timer and retransmit the message when the timer has expired.

The structure of the *handshake* message header is shown in Figure 5.12.

The *Message Sequence Number* field, coded on 2 bytes, contains the number of the *handshake* messages. The value of this field is constant for each message fragment.

The *Fragment Offset* field, coded on 3 bytes, indicates the total number of bytes in all previous fragments. The value of this field is zero if the message is not fragmented.

The *Fragment Length* field, coded on 3 bytes, indicates the size of the fragment. The value of this field is equal to the value of the *Message Length* field if the message is not fragmented.

Protocol	Version	Epoch...
...Epoch	Sequence Number....	
... Sequence Number		Length ..
...Length	Type	Message Length...
Message Length	Message Sequence Number	Fragment Offset...
...Fragment Offset		Fragment Length...
...Fragment Length	Message / Fragment	

Figure 5.12. *Handshake message header in DTLS protocol*

5.3.2. Adaptation to DCCP transport

The DCCP protocol has the following characteristics:

- as for the TCP protocol, the transmission of data is contingent on the opening of a session between the client and the server;

- as for the UDP protocol, the data transmitted is not subject to retransmission in the event of loss;
- the data transmitted is the subject of an acknowledgment, the purpose of which is not the retransmission of missing data but the provision of information on data delivered to the destination;
- the protocol's functionalities are the subject of a negotiation;
- the congestion checking mechanism is chosen by the application layer.

As for TLS with regard to TCP, the DCCP connection is made before the *handshake* procedure of the DTLS protocol. However, it is possible to transmit data during a DCCP connection. In this case, the data transmitted is composed of the *handshake* procedure messages.

5.3.3. Adaption to SCTP transport

The SCTP protocol has the following characteristics:

- an association is initialized by the client. During the initialization phase, a reference (*cookie*) is implemented for protection against attacks.
- as for the TCP protocol, data is delivered to the application without error and in sequence. If the transfer executed by the TCP protocol is byte-oriented, the transfer executed by the SCTP protocol is block oriented.
- as for the TCP protocol, the source output is regulated either by the receiver or by the *Slow-Start* and *Congestion Avoidance* congestion control mechanism;
- fragmentation is ensured by the transmitter to adapt to the minimum size encountered in the network;

- messages from multiple applications can be multiplexed in a single SCTP connection;
- multiple interfaces per host can participate in association (*multi-homing*).

Though the SCTP protocol functions like the TCP protocol, it uses the DTLS protocol rather than the TLS protocol. This is because the SCTP protocol has specific functionalities that are not compatible with the TLS protocol:

- the SCTP protocol can transport messages that are not in sequence;
- the SCTP protocol can transport messages without an acknowledgment;
- the SCTP protocol can transport one-way messages.

Some functions executed by the DTLS protocol are removed when they are already executed by the SCTP protocol:

- retransmission in case of loss;
- finding of the maximum MTU value.

5.3.4. Adaption to SRTP transport

The Real-time Transport Protocol (RTP) is the protocol used by real-time applications (voice and video). It is a complementary transport protocol to the associated UDP protocol. It provides the following functions:

- identification of type of multimedia information;
- numbering of RTP segments;
- time-stamping of RTP segments.

The RTP Control Protocol (RTCP) is the control protocol associated with the RTP protocol. RTCP messages provide the following information:

- service quality parameter values such as loss rate, delay and jitter of RTP segments;
- transmitter identifier, clock for synchronizing two data sources (voice and video), cumulative segment counter and number of bytes sent.

The SRTP protocol encrypts the data encapsulated by the RTP header and RTCP messages, as well as checking the integrity of RTP segments or RTCP messages.

The DTLS protocol provides a frame for the SRTP protocol for security parameter negotiation. However, the encryption and integrity-checking principles defined for the SRTP protocol are maintained, meaning that RTP or RTCP flows are not encapsulated by the *Record* header.

The *handshake* procedure is part of the Session Information Protocol (SIP) procedure used to establish a session (for example, a telephone call). When the RTP flow (containing the voice, for example) has been negotiated via the Session Description Protocol (SDP), the SIP party initializes the sending of the first *client_echo* message of the *handshake* procedure.

The *client_hello* message contains the security parameters proposed by the client. These parameters, specific to the RTP flow, are defined in a specific extension called *use_srtp*. Several profiles are available, specifying the encryption and integrity checking algorithms.

The server's *hello_server* response contains the security parameter finally chosen by the server in the *use_srtp* extension.